

```
#!/usr/bin/env python3
"""
=====
=====
=====

  BINANCE FUTURES MONITOR BOT - OI
+ Volumen (WebSocket + REST
híbrido)
=====
=====
=====

Arquitectura:
  - WebSocket
(wss://fstream.binance.com) ->
velas de 1 minuto en tiempo real
  para detectar spikes de
volumen y calcular RVOL (sin
gastar peso REST).
  - REST API pública -> Open
Interest, consultado en bucle
secuencial con
  pausa fija entre requests
para evitar rate limits (HTTP
```

418/429).

- Telegram con DOS canales: uno urgente (con sonido) para spikes de

- volumen explosivos, y uno informativo (silencioso) para OI y RVOL.

No usa API Keys privadas: solo endpoints públicos de Binance Futuros.

No es asesoría financiera. Uso bajo tu propia responsabilidad.

```
=====
=====
=====
"""
```

```
import os
import json
import time
import logging
import threading
from collections import deque
from http.server import
BaseHTTPRequestHandler,
```

HTTPServer

```
import requests
import websocket # pip install
websocket-client

#
=====
=====
=====
# ===== BLOQUE DE
CONFIGURACIÓN (EDITAR AQUÍ)
=====
#
=====
=====
=====
class Config:
    # -----
    -----
    -----
    # TELEGRAM – credenciales del
    bot y de los DOS canales de
    alertas.
    # Se recomienda definirlas
```

como variables de entorno en el
servidor

(Render, etc.) en vez de
escribirlas directamente aquí.

TELEGRAM_TOKEN =
os.environ.get("TELEGRAM_TOKEN",
"PON_TU_TOKEN_AQUI")

Canal URGENTE: spikes de
volumen explosivos en velas de
1m. Suena.

CHAT_ID_URGENTE =
os.environ.get("CHAT_ID_URGENTE",
"PON_TU_CHAT_ID_URGENTE")

Canal INFO: monitor general
de Open Interest y RVOL.
Silencioso.

CHAT_ID_INFO =
os.environ.get("CHAT_ID_INFO",
"PON_TU_CHAT_ID_INFO")

```
-----  
-----  
    # BINANCE – endpoints base  
(públicos, sin API Key)  
    # -----  
-----  
-----  
    REST_BASE =  
"https://fapi.binance.com"  
    WS_BASE =  
"wss://fstream.binance.com/stream"  
"  
  
    # -----  
-----  
-----  
    # FILTRADO DINÁMICO DE PARES  
    # -----  
-----  
-----  
    TOP_N_PAIRS =  
200                # cuántos  
pares monitorear como máximo  
    MIN_VOLUME_USD =  
10_000_000        # descarta pares  
con menos de $10M vol 24h
```

```

        SYMBOL_REFRESH_INTERVAL = 6 *
60 * 60    # refrescar lista de
pares cada 6h

        # -----
-----
-----

        # OPEN INTEREST – frecuencia
de consulta y pausa anti rate-
limit
        # -----
-----
-----

        OI_CHECK_INTERVAL = 5 *
60          # cada cuánto se
recorren TODOS los pares
        OI_REQUEST_PAUSE =
0.08        # pausa entre cada
request de OI (segundos)
        OI_HISTORY_MAXLEN =
300         # ~25h de histórico
a razón de 1 muestra/5min

        # Umbrales de variación de
OI, evaluados de forma
INDEPENDIENTE por

```

```
# temporalidad. Si se cumple
cualquiera de ellos, se dispara
la alerta
```

```
# indicando cuál(es)
temporalidad(es) la activaron.
```

```
OI_THRESHOLD_5M = 3.0
```

```
OI_THRESHOLD_15M = 5.0
```

```
OI_THRESHOLD_30M = 7.0
```

```
OI_THRESHOLD_1H = 10.0
```

```
OI_THRESHOLD_4H = 15.0
```

```
OI_THRESHOLD_24H = 20.0
```

```
# -----
```

```
-----
# DETECTOR DE SPIKES DE
VOLUMEN EN VELAS DE 1 MINUTO
(canál urgente)
```

```
# -----
```

```
-----
VOL_SPIKE_THRESHOLD_PCT =
300      # % de exceso sobre el
promedio para alertar
```

```
LOOKBACK_BARS_1M =
20          # nº de velas
```

previas usadas como referencia

```
# -----  
-----  
-----  
# DETECTOR DE RVOL (Volumen  
Relativo) – canal info  
# -----  
-----  
-----  
RVOL_WINDOW_BARS =  
5           # tamaño de la  
ventana acumulada (minutos)  
RVOL_THRESHOLD =  
3.0         # dispara si  
RVOL >= 3.0x (300%) lo esperado  
  
# -----  
-----  
-----  
# ANTI-SPAM (cooldowns  
independientes por tipo de  
alerta)  
# -----  
-----  
-----
```

```
SPIKE_ALERT_COOLDOWN = 15 *  
60  
OI_ALERT_COOLDOWN = 15 * 60  
RVOL_ALERT_COOLDOWN = 15 * 60  
  
# -----  
-----  
-----  
# MARKET CAP (CoinGecko – API  
pública, sin API Key)  
# -----  
-----  
-----  
# Binance no expone market  
cap (ni REST ni WebSocket), así  
que se  
# obtiene de CoinGecko y se  
cachea en memoria. El match  
símbolo->moneda  
# es CONSERVADOR: solo se  
acepta si el ticker es único  
entre las monedas  
# descargadas; si hay  
ambigüedad (varios proyectos con  
el mismo ticker)  
# o no se encuentra, esa
```

alerta simplemente omite la línea
de Market Cap

(nunca detiene ni rompe el
bot).

ENABLE_MARKET_CAP = True

COINGECKO_BASE =

"https://api.coingecko.com/api/v3
"

MARKET_CAP_REFRESH_INTERVAL =
45 * 60 # refrescar cache cada
45 min

MARKET_CAP_PAGES =

4 # 4 páginas
x 250 = top 1000 por market cap

MARKET_CAP_PAGE_SIZE = 250

MARKET_CAP_REQUEST_PAUSE =

1.5 # pausa entre
páginas (cortesía API pública)

RED / ROBUSTEZ

```
REQUEST_TIMEOUT = 10
MAX_RETRIES = 3
RETRY_BACKOFF_BASE = 2

# Servidor HTTP mínimo
(necesario para plataformas tipo
Render que
    # exigen un puerto abierto en
servicios "Web Service" + keep-
alive)
    ENABLE_HEALTH_SERVER = True

# Temporalidades de OI a evaluar:
nombre -> (segundos, umbral %,
tolerancia seg.)
OI_TIMEFRAMES = {
    "5m": (300,
Config.OI_THRESHOLD_5M, 90),
    "15m": (900,
Config.OI_THRESHOLD_15M, 180),
    "30m": (1800,
Config.OI_THRESHOLD_30M, 300),
    "1h": (3600,
Config.OI_THRESHOLD_1H, 450),
    "4h": (14400,
```

```

Config.OI_THRESHOLD_4H, 900),
    "24h": (86400,
Config.OI_THRESHOLD_24H, 1800),
}

#
=====
=====
=====
# LOGGING
#
=====
=====
=====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%
(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log =
logging.getLogger("futures_monito
r")
logging.getLogger("websocket").se
tLevel(logging.WARNING)

```

```

#
=====
=====
=====
# ESTADO GLOBAL EN MEMORIA
#
=====
=====
=====
http_session = requests.Session()
http_session.headers.update({"User-Agent": "futures-monitor-bot/2.0"})

symbols_lock = threading.Lock()
active_symbols =
[]                # Top N pares
filtrados (símbolos en
mayúsculas)

oi_history =
{}                # symbol -
> deque[(ts, oi_usdt)]
kline_volume_history =

```

```
{ }          # symbol ->
deque[quote_volume de velas
cerradas]
kline_lock = threading.Lock()

last_spike_alert =
{ }          # symbol -> ts
último alert de spike
last_oi_alert =
{ }          # symbol ->
ts último alert de OI
last_rvol_alert =
{ }          # symbol -> ts
último alert de RVOL

current_ws_app =
None          # referencia
al WebSocketApp activo
ws_restart_lock =
threading.Lock()

market_cap_cache =
{ }          # base_symbol
(ej. "QNT") -> market cap USD
market_cap_lock =
threading.Lock()
```

```

#
=====
=====
=====
# HELPERS GENERALES
#
=====
=====
=====
def format_usd(value):
    if value >= 1_000_000_000:
        return f"${value /
1_000_000_000:.2f}B"
    if value >= 1_000_000:
        return f"${value /
1_000_000:.2f}M"
    if value >= 1_000:
        return f"${value /
1_000:.2f}K"
    return f"${value:.2f}"

def tradingview_link(symbol):
    """Enlace directo a

```

```
TradingView para el perpetuo de
Binance."""
    return
    f"https://es.tradingview.com/chart/?symbol=BINANCE:{symbol}.P"

def base_asset(symbol):
    """Convierte un símbolo de
    futuros (ej. 'QNTUSDT') en su
    activo base ('QNT')."""
    return symbol[:-4] if
    symbol.endswith("USDT") else
    symbol

def get_market_cap(symbol):
    """Devuelve el market cap
    cacheado para un símbolo, o None
    si no hay
    match único/confiable en
    CoinGecko (nunca lanza error,
    nunca bloquea)."""
    with market_cap_lock:
        return
    market_cap_cache.get(base_asset(s
```

```
ymbol))
```

```
def http_get_json(url,
params=None):
    """GET con reintentos y
backoff exponencial. Devuelve
None si falla todo."""
    for attempt in range(1,
Config.MAX_RETRIES + 1):
        try:
            resp =
http_session.get(url,
params=params,
timeout=Config.REQUEST_TIMEOUT)
            if resp.status_code
== 200:
                return
resp.json()
            if resp.status_code
in (429, 418):
                wait =
Config.RETRY_BACKOFF_BASE **
attempt
                log.warning("Rate
limit Binance (status %s).
```

```

Esperando %ss...",
resp.status_code, wait)
        time.sleep(wait)
    else:

log.warning("Respuesta inesperada
(status %s) en %s",
resp.status_code, url)

time.sleep(Config.RETRY_BACKOFF_B
ASE)
        except
requests.exceptions.RequestExcept
ion as e:
            wait =
Config.RETRY_BACKOFF_BASE **
attempt
            log.warning("Error de
red (%s) en %s [intento %s/%s].
Reintentando en %ss...",
                        e, url,
attempt, Config.MAX_RETRIES,
wait)
            time.sleep(wait)
            log.error("Fallaron todos los
reintentos para %s", url)

```

```
return None
```

```
def find_closest_sample(history,
target_seconds_ago, tolerance):
    """Busca en el histórico la
muestra más cercana a
'target_seconds_ago'
dentro de una tolerancia
(segundos). Devuelve (ts, valor)
o None."""
    now = time.time()
    target_ts = now -
target_seconds_ago
    best, best_diff = None, None
    for ts, value in history:
        diff = abs(ts -
target_ts)
        if diff <= tolerance and
(best_diff is None or diff <
best_diff):
            best, best_diff =
(ts, value), diff
    return best
```

```

#
=====
=====
=====
# TELEGRAM (doble canal)
#
=====
=====
=====
def send_telegram(text, chat_id,
silent=False):
    if "PON_TU" in
Config.TELEGRAM_TOKEN or "PON_TU"
in str(chat_id):
        log.warning("Telegram no
configurado (token/chat_id por
defecto). Mensaje no enviado.")
        return False

    url =
f"https://api.telegram.org/bot{Co
nfig.TELEGRAM_TOKEN}/sendMessage"
    payload = {
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "Markdown",

```

```
"disable_web_page_preview": True,
    "disable_notification":
silent,
    }
    try:
        resp =
http_session.post(url,
json=payload,
timeout=Config.REQUEST_TIMEOUT)
        if resp.status_code !=
200:
            log.error("Error
enviando a Telegram: %s - %s",
resp.status_code, resp.text)
            return False
        return True
    except
requests.exceptions.RequestExcept
ion as e:
        log.error("Excepción
enviando a Telegram: %s", e)
        return False

def send_urgent_alert(text):
```

```

        return send_telegram(text,
Config.CHAT_ID_URGENTE,
silent=False)

def send_info_alert(text):
    return send_telegram(text,
Config.CHAT_ID_INFO, silent=True)

#
=====
=====
=====
# BINANCE REST: FILTRADO DINÁMICO
DE PARES
#
=====
=====
=====
def fetch_exchange_info():
    data = http_get_json(f"
{Config.REST_BASE}/fapi/v1/exchan
geInfo")
    if not data:
        return None

```

```

        symbols = set()
        for s in data.get("symbols",
[]):
            if (
                s.get("quoteAsset")
== "USDT"
                and
s.get("contractType") ==
"PERPETUAL"
                and s.get("status")
== "TRADING"
            ):

symbols.add(s["symbol"])
        return symbols

def fetch_24h_tickers():
    data = http_get_json(f"
{Config.REST_BASE}/fapi/v1/ticker
/24hr")
    if not data:
        return None
    result = {}
    for item in data:
        try:

```

```
result[item["symbol"]] = {
    "price":
float(item["lastPrice"]),
    "quote_volume":
float(item["quoteVolume"]),
}
except (KeyError,
ValueError, TypeError):
    continue
return result
```

```
def refresh_symbols():
    """Actualiza la lista global
de pares (Top N por volumen, >=
MIN_VOLUME_USD).
    Si la lista cambia, fuerza la
reconexión del WebSocket con los
nuevos streams."""
    log.info("Refrescando lista
de pares (exchangeInfo + volumen
24h)...")

    valid_symbols =
fetch_exchange_info()
```

```

        tickers = fetch_24h_tickers()
        if not valid_symbols or not
tickers:
            log.error("No se pudo
refrescar la lista de pares. Se
mantiene la anterior.")
            return

        candidates = [
            (s, tickers[s]
["quote_volume"])
            for s in valid_symbols
            if s in tickers and
tickers[s]["quote_volume"] >=
Config.MIN_VOLUME_USD
        ]
        candidates.sort(key=lambda x:
x[1], reverse=True)
        new_symbols = [s for s, _ in
candidates[: Config.TOP_N_PAIRS]]

        with symbols_lock:
            global active_symbols
            changed =
set(new_symbols) !=
set(active_symbols)

```

```

        active_symbols =
new_symbols

        log.info("Lista de pares
actualizada: %s pares activos (de
%s candidatos con volumen >=
%s).",
                len(new_symbols),
len(candidates),
format_usd(Config.MIN_VOLUME_USD)
)

        if changed:
            restart_websocket()

#
=====
=====
=====
# COINGECKO: MARKET CAP (cache en
memoria, refresco periódico)
#
=====
=====
=====

```

```

def fetch_market_caps():
    """Descarga varias páginas de
    CoinGecko (ordenadas por market
    cap) y arma
        un diccionario símbolo ->
    market cap, aceptando SOLO los
    tickers únicos
        (si un mismo ticker aparece
    en más de un proyecto, se
    descarta para evitar
        asignar el market cap
    equivocado)."""
    symbol_entries = {} #
    símbolo -> lista de market caps
    encontrados (para detectar
    duplicados)

    for page in range(1,
    Config.MARKET_CAP_PAGES + 1):
        data = http_get_json(
            f"
{Config.COINGECKO_BASE}/coins/mar
kets",
            params={
                "vs_currency":
"usd",

```

```

        "order":
"market_cap_desc",
        "per_page":
Config.MARKET_CAP_PAGE_SIZE,
        "page": page,
        "sparkline":
"false",
    },
)
if not data:

log.warning("CoinGecko: no se
pudo obtener la página %s de
market caps.", page)
    continue

    for item in data:
        try:
            sym =
item["symbol"].upper()
            mcap =
item.get("market_cap")
            if mcap:

symbol_entries.setdefault(sym,
[]).append(mcap)

```

```
                except (KeyError,
TypeError):
                    continue

time.sleep(Config.MARKET_CAP_REQUIRE_PAUSE) # cortesía con la API pública

        # Solo se aceptan tickers UNICOS entre todo lo descargado (match conservador)
        unique_map = {sym: caps[0]
for sym, caps in
symbol_entries.items() if
len(caps) == 1}
        ambiguous_count = sum(1 for
caps in symbol_entries.values()
if len(caps) > 1)
        if ambiguous_count:
            log.info("CoinGecko: %s
tickers ambiguos descartados
(varios proyectos comparten
símbolo).",
ambiguous_count)
```

```
        return unique_map

def refresh_market_caps():
    if not
Config.ENABLE_MARKET_CAP:
        return
    log.info("Refrescando cache
de Market Cap (CoinGecko)...")
    new_map = fetch_market_caps()
    if not new_map:
        log.warning("No se pudo
refrescar Market Cap. Se mantiene
el cache anterior.")
        return
    with market_cap_lock:
        global market_cap_cache
        market_cap_cache =
new_map
    log.info("Cache de Market Cap
actualizado: %s símbolos con
match único.", len(new_map))

def run_market_cap_refresher():
    """Hilo de fondo: refresca el
```

```

cache de Market Cap cada
MARKET_CAP_REFRESH_INTERVAL."""
    while True:
        try:
            refresh_market_caps()
        except Exception as e:
            log.exception("Error
refrescando Market Cap: %s", e)

    time.sleep(Config.MARKET_CAP_REFRESH_INTERVAL)

#
=====
=====
=====
# BINANCE REST: OPEN INTEREST
(bucle secuencial con pausa anti
rate-limit)
#
=====
=====
=====
def fetch_open_interest(symbol):
    data = http_get_json(f"

```

```

{Config.REST_BASE}/fapi/v1/openInterest", params={"symbol":
symbol})
    if not data:
        return None
    try:
        return
float(data["openInterest"])
    except (KeyError, ValueError,
TypeError):
        return None

```

```

def
evaluate_oi_timeframes(symbol,
hist, oi_usdt):
    """Evalúa el cambio de OI
contra CADA temporalidad de forma
independiente.
    Solo dispara ante SUBIDAS de
OI (acumulación) – las caídas se
ignoran a
    propósito, ya que el objetivo
es detectar entradas de dinero,
no salidas.
    Devuelve una lista de tuplas

```

```

(label, pct_change) de las que
superaron su umbral."""
    triggered = []
    for label, (secs, threshold,
tolerance) in
OI_TIMEFRAMES.items():
        sample =
find_closest_sample(hist, secs,
tolerance)
        if sample and sample[1] >
0:
            change = (oi_usdt -
sample[1]) / sample[1] * 100
            if change >=
threshold: # solo variación
POSITIVA (acumulación de OI)

triggered.append((label, change))
    return triggered

def process_symbol_oi(symbol,
price, quote_volume):
    oi_contracts =
fetch_open_interest(symbol)
    if oi_contracts is None:

```

```

        return

        oi_usdt = oi_contracts *
price
        now = time.time()
        hist =
oi_history.setdefault(symbol,
deque(maxlen=Config.OI_HISTORY_MA
XLEN))

        triggered =
evaluate_oi_timeframes(symbol,
hist, oi_usdt)
        hist.append((now, oi_usdt))
# guardar DESPUÉS de comparar,
para no compararse consigo mismo

        if not triggered:
            return

        last_sent =
last_oi_alert.get(symbol, 0)
        if now - last_sent <
Config.OI_ALERT_COOLDOWN:
            return

```

```

# Todas las alertas triggered
son subidas (ver
evaluate_oi_timeframes), así
# que el marcador siempre es
verde.

lines = "\n".join(f"• *
{label}:* +{chg:.2f}%" for
label, chg in triggered)

mcap = get_market_cap(symbol)
mcap_line = f"*Market Cap:*
{format_usd(mcap)}\n" if mcap
else ""

msg = (
    f" *ALERTA DE OPEN
INTEREST (Acumulación)*\n\n"
    f"*Par:* #{symbol}\n"
    f"*Temporalidad(es)
activada(s):*\n{lines}\n\n"
    f"*OI Actual:*
{format_usd(oi_usdt)}\n"
    f"{mcap_line}"
    f"*Volumen 24h:*
{format_usd(quote_volume)}\n\n"
    f" [Ver en TradingView]

```

```

({tradingview_link(symbol)}})"
    )
    if send_info_alert(msg):
        last_oi_alert[symbol] =
now
        log.info("Alerta OI
enviada -> %s | %s", symbol, [l
for l, _ in triggered])

def check_oi_cycle():
    """Recorre TODOS los pares
    activos de forma SECUENCIAL, con
    una pausa fija
    entre cada request para
    evitar bloqueos/HTTP 418 de
    Binance."""
    with symbols_lock:
        symbols_snapshot =
list(active_symbols)

    if not symbols_snapshot:
        log.warning("No hay pares
        activos todavía. Saltando ciclo
        de OI.")
    return

```

```
        tickers = fetch_24h_tickers()
        if not tickers:
            log.error("No se pudo
obtener ticker/24hr. Saltando
ciclo de OI.")
            return

        start = time.time()
        processed = 0
        for symbol in
symbols_snapshot:
            t = tickers.get(symbol)
            if not t:
                continue
            try:

process_symbol_oi(symbol,
t["price"], t["quote_volume"])
                processed += 1
            except Exception as e:
                log.exception("Error
procesando OI de %s: %s", symbol,
e)

        time.sleep(Config.OI_REQUEST_PAUS
```

```
E) # <-- pausa anti rate-limit
```

```
    log.info("Ciclo de OI  
completado: %s/%s pares  
procesados en %.1fs.",  
            processed,  
len(symbols_snapshot),  
time.time() - start)
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
# WEBSOCKET: VELAS DE 1 MINUTO  
(spikes de volumen + RVOL)
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
def build_stream_url(symbols):  
    streams = "/".join(f"  
{s.lower()}@kline_1m" for s in  
symbols)  
    return f"{Config.WS_BASE}?  
streams={streams}"
```

```
def evaluate_volume_bar(symbol,
quote_volume):
    """Corrige el bug de la media
    siempre en ~1.01x: usa el
    histórico REAL de
        barras cerradas guardado en
    memoria (deque), calculado ANTES
    de insertar
        la barra actual, en vez de
    una media móvil actualizada
    incorrectamente."""
    with kline_lock:
        hist =
kline_volume_history.setdefault(
            symbol,
deque(maxlen=max(Config.LOOKBACK_
BARS_1M, Config.RVOL_WINDOW_BARS)
+ 5)
        )
        hist_snapshot =
list(hist) # copia para no
bloquear mientras evaluamos

        now = time.time()
```



```

f"*Par:*
#{symbol}\n"

f"*Volumen última vela:*
{format_usd(quote_volume)}\n"

f"*Promedio
({Config.LOOKBACK_BARS_1M}
velas):* {format_usd(avg_vol)}\n"

f"*Exceso:* +
{spike_pct:.0f}%\n\n"
f" [Ver
en TradingView]
({tradingview_link(symbol)})"
)
if
send_urgent_alert(msg):

last_spike_alert[symbol] = now

log.info("Alerta SPIKE enviada ->
%s | +%.0f%%", symbol, spike_pct)

# -----
RVOL DETECTOR (canal info) -----

```

```

-----
        if len(hist_snapshot)
>= Config.RVOL_WINDOW_BARS - 1:
            window =
hist_snapshot[-
(Config.RVOL_WINDOW_BARS - 1):] +
[quote_volume]
            actual_sum =
sum(window)
            expected_sum =
avg_vol * Config.RVOL_WINDOW_BARS
            if expected_sum >
0:
                rvol =
actual_sum / expected_sum
                if rvol >=
Config.RVOL_THRESHOLD:

last_sent_rvol =
last_rvol_alert.get(symbol, 0)
                if now -
last_sent_rvol >=
Config.RVOL_ALERT_COOLDOWN:
                    msg =
(

```

```

f"📊 *RVOL ANÓMALO*\n\n"

f"*Par:* #{symbol}\n"

f"*RVOL:* {rvol:.2f}x\n"

f"*Volumen acumulado
({Config.RVOL_WINDOW_BARS}m):*
{format_usd(actual_sum)}\n"

f"*Volumen esperado:*
{format_usd(expected_sum)}\n\n"

f"📊 [Ver en TradingView]
({tradingview_link(symbol)})"
)
if
send_info_alert(msg):

last_rvol_alert[symbol] = now

log.info("Alerta RVOL enviada ->
%s | %.2fx", symbol, rvol)

    with kline_lock:
        hist.append(quote_volume)

```

```
def on_ws_message(ws, message):
    try:
        payload =
        json.loads(message)
        data =
        payload.get("data", {})
        if data.get("e") !=
        "kline":
            return
        k = data.get("k", {})
        if not k.get("x"): #
        solo procesar velas CERRADAS
            return
        symbol = k.get("s")
        quote_volume =
        float(k.get("q", 0))
        if symbol:

        evaluate_volume_bar(symbol,
        quote_volume)
    except Exception as e:
        log.exception("Error
        procesando mensaje de WebSocket:
        %s", e)
```

```
def on_ws_error(ws, error):  
    log.warning("WebSocket error:  
%s", error)
```

```
def on_ws_close(ws,  
close_status_code, close_msg):  
    log.warning("WebSocket  
cerrado (code=%s, msg=%s). Se  
reconectará.", close_status_code,  
close_msg)
```

```
def on_ws_open(ws):  
    log.info("WebSocket conectado  
correctamente.")
```

```
def restart_websocket():  
    """Fuerza el cierre del  
WebSocket activo para que el hilo  
lo reconstruya  
    con la lista de símbolos  
actualizada."""
```

```
        with ws_restart_lock:
            global current_ws_app
            if current_ws_app is not
None:
                try:

current_ws_app.close()
                    except Exception:
                        pass


def run_websocket_forever():
    """Hilo de fondo: mantiene el
WebSocket vivo y lo reconecta
ante cualquier
    caída o cambio en la lista de
símbolos monitoreados."""
    global current_ws_app
    while True:
        with symbols_lock:
            symbols_snapshot =
list(active_symbols)

            if not symbols_snapshot:
                time.sleep(5)
                continue
```

```
        url =
build_stream_url(symbols_snapshot
)

        log.info("Conectando
WebSocket con %s streams de velas
1m...", len(symbols_snapshot))

        ws_app =
websocket.WebSocketApp(
            url,
            on_open=on_ws_open,

on_message=on_ws_message,
            on_error=on_ws_error,
            on_close=on_ws_close,
        )
        current_ws_app = ws_app

        try:

ws_app.run_forever(ping_interval=
180, ping_timeout=10)
            except Exception as e:

log.exception("Excepción en
```

```

WebSocket run_forever: %s", e)

        time.sleep(5) # pequeño
        backoff antes de reconectar

#
=====
=====
=====
# SERVIDOR HTTP MÍNIMO (keep-
  alive para Render u otras
  plataformas)
#
=====
=====
=====
class
HealthHandler(BaseHTTPRequestHand
ler):
    def do_GET(self):
        self.send_response(200)

self.send_header("Content-type",
"text/plain")
        self.end_headers()

```

```
        with symbols_lock:
            n =
len(active_symbols)
            self.wfile.write(f"OK -
Futures Monitor Bot activo. Pares
monitoreados: {n}".encode())

        def log_message(self, format,
*args):
            pass

def start_health_server():
    port =
int(os.environ.get("PORT",
10000))
    server =
HTTPServer(("0.0.0.0", port),
HealthHandler)
    log.info("Servidor de salud
escuchando en el puerto %s (para
keep-alive).", port)
    server.serve_forever()

#
```

```

=====
=====
=====
# LOOP PRINCIPAL
#
=====
=====
=====
def main():
    log.info("=== Iniciando
Binance Futures Monitor Bot
(WebSocket + REST) ===")
    log.info(
        "Spike: >=%.0f%% sobre %s
velas | RVOL: >=%.1fx sobre %s
velas | "
        "OI umbrales
5m/15m/30m/1h/4h/24h =
%.1f/%.1f/%.1f/%.1f/%.1f/%.1f
%%",
        Config.VOL_SPIKE_THRESHOLD_PCT,
        Config.LOOKBACK_BARS_1M,
        Config.RVOL_THRESHOLD,
        Config.RVOL_WINDOW_BARS,
        Config.OI_THRESHOLD_5M,

```

```
Config.OI_THRESHOLD_15M,  
Config.OI_THRESHOLD_30M,  
        Config.OI_THRESHOLD_1H,  
Config.OI_THRESHOLD_4H,  
Config.OI_THRESHOLD_24H,  
    )  
  
    if  
Config.ENABLE_HEALTH_SERVER:  
  
    threading.Thread(target=start_health_server, daemon=True).start()  
  
        refresh_symbols() # primera  
carga de pares (dispara conexión  
WS más abajo)  
  
    threading.Thread(target=run_websocket_forever,  
daemon=True).start()  
  
    if Config.ENABLE_MARKET_CAP:  
  
    threading.Thread(target=run_market_cap_refresher,
```

```
daemon=True).start()

    send_info_alert("✅ *Futures  
Monitor Bot iniciado  
correctamente.*")

    last_symbol_refresh =  
time.time()

    last_oi_check = 0 # forzar  
primera ejecución inmediata

    while True:
        try:
            now = time.time()

            if now -  
last_symbol_refresh >=  
Config.SYMBOL_REFRESH_INTERVAL:  
                refresh_symbols()

            last_symbol_refresh = now

            if now -  
last_oi_check >=  
Config.OI_CHECK_INTERVAL:  
                check_oi_cycle()
```

```
        last_oi_check =  
time.time()  
  
        except Exception as e:  
            log.exception("Error  
inesperado en el loop principal:  
%s", e)  
  
            time.sleep(10)  
  
if __name__ == "__main__":  
    try:  
        main()  
    except KeyboardInterrupt:  
        log.info("Bot detenido  
manualmente.")
```